UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN SOFTWARE

ASIGNATURA:

Aplicaciones Distribuidas

TEMA:

Informe Académico Individual: Análisis Crítico Comparativo de Elasticsearch y Apache Solr como Sistemas Distribuidos

PRESENTADO POR:

HERRERA BARCO HUMBERTO ALDAIR

FECHA:

13/11/2026

Índice

Resumen **4**

1. Introducción **4**
1.1. Problema que resuelve el sistema 4
1.2. Uso y escala 4
1.3. Motivación de la elección 5

2. Arquitectura del sistema **5**
2.1. Componentes principales de Elasticsearch 5
2.2. Comunicación entre componentes 5
2.3. Componentes principales de Apache Solr (SolrCloud) 6
2.4. Diagrama de arquitectura 6
2.5. Evolución desde el esquema a mano (Anexo A) 6

3. Análisis con los conceptos de la Unidad 1 **7**
3.1. Transparencias de distribución 7
3.2. Teorema CAP y extensión PACELC 8
3.2.1. Comportamiento ante partición de red (CAP) 8
3.2.2. Comportamiento sin partición (PACELC) 8
3.3. Modelos de comunicación 8
3.4. Tolerancia a fallos 9
3.4.1. Fallos soportados, detección y recuperación 9
3.4.2. Evidencia desde la experimentación 9
3.4.3. Incidente real documentado 9

4. Experimentación propia **10**
4.1. Levantamiento del clúster 10
4.2. Creación de índice y distribución de shards 11
4.3. Caída de un nodo 11
4.4. Recuperación del nodo 12

5. Ejemplos del entorno del estudiante **13**
5.1. Ejemplo 1: [Completar: nombre del sistema observado] 13
5.2. Ejemplo 2: [Completar: nombre del sistema observado] 13

6. Análisis comparativo: Elasticsearch vs. Apache Solr **14**
6.1. Discusión comparativa 15

7. Postura crítica **15**
7.1. Fortalezas 15
7.2. Limitaciones y compromisos (trade-offs) 15
7.3. Contextos donde no sería la mejor opción 16

8. Conclusiones **16**

9. Declaración de uso de inteligencia artificial **16**

Referencias **17**

A. Esquema inicial a mano de la arquitectura **19**

B. Capturas de lectura de las fuentes **19**
B.1. Fuente 1: Comparación de Elasticsearch y Apache Solr [4] 20
B.2. Fuente 2: Plataforma distribuida con Elasticsearch y Kafka [2] 20
B.3. Fuente 3: Uso de Solr y evaluación de rendimiento [6] 21

Resumen

Este informe analiza críticamente a Elasticsearch como sistema distribuido real y lo compara con Apache Solr (SolrCloud), un motor de búsqueda y análisis basado en Apache Lucene que resuelve problemas similares de indexación y consulta de grandes volúmenes de información. El estudio aplica conceptos de la Unidad 1: transparencias de distribución, teorema CAP y extensión PACELC, modelos de comunicación y tolerancia a fallos. La metodología combina la revisión de artículos académicos recientes y documentación técnica oficial con una experimentación propia sobre un clúster local de Elasticsearch de tres nodos desplegado con Docker Compose. Durante las pruebas se creó un índice con tres shards primarios y una réplica, se verificó la distribución de fragmentos y se evaluó el comportamiento ante la caída y recuperación del nodo `es02`. Los resultados mostraron que el clúster mantuvo el estado *green* y conservó los shards activos mediante la reasignación entre los nodos disponibles, evidenciando mecanismos de replicación y recuperación. La comparación con SolrCloud permitió identificar similitudes arquitectónicas y diferencias en la gestión de réplicas, coordinación y compromisos de consistencia, disponibilidad y latencia. Se concluye que Elasticsearch ofrece una solución distribuida adecuada para búsqueda y análisis escalables, aunque su operación exige comprender sus configuraciones y límites ante particiones y fallos.

1. Introducción

1.1. Problema que resuelve el sistema

El crecimiento del volumen de datos textuales y estructurados ha hecho necesario contar con sistemas capaces de indexar y buscar información de forma eficiente. Elasticsearch es un motor de búsqueda y análisis distribuido orientado al almacenamiento, indexación y consulta de grandes volúmenes de datos, lo que lo hace útil en escenarios como búsqueda de texto completo, análisis de registros y observación de plataformas [1], [2], [3]. Apache Solr es una alternativa basada también en Apache Lucene, ampliamente utilizada para búsqueda e indexación distribuida; su uso en casos reales y estudios comparativos permite contrastar capacidades y decisiones de diseño frente a Elasticsearch [4], [5], [6].

1.2. Uso y escala

Elasticsearch se emplea en aplicaciones donde se requiere consultar y analizar datos de manera rápida sobre conjuntos distribuidos, apoyándose en una arquitectura de clúster que permite escalar horizontalmente mediante nodos y partición de índices en shards [2], [7], [8]. En el caso de Solr, su modo distribuido (SolrCloud) permite repartir colecciones en shards y réplicas para atender cargas de búsqueda e indexación [4], [9]. La elección de uno

u otro sistema depende de factores como el tipo de carga, los requisitos de consistencia, la tolerancia a fallos, el rendimiento y la operación del clúster.

1.3. Motivación de la elección

Se eligió Elasticsearch por la posibilidad de desplegar un clúster local con Docker Compose y observar de manera práctica conceptos de sistemas distribuidos: formación del clúster, partición de datos en shards, replicación, y reacción ante la caída y recuperación de un nodo. La comparación con Apache Solr se consideró pertinente porque ambos resuelven el problema de la búsqueda distribuida y permiten analizar compromisos de diseño distintos dentro del mismo dominio [4].

2. Arquitectura del sistema

2.1. Componentes principales de Elasticsearch

Un clúster de Elasticsearch está compuesto por uno o más nodos, donde cada nodo es una instancia del servidor que participa en el almacenamiento de datos y en la ejecución de operaciones de indexación y búsqueda [7], [8]. Los datos se organizan en índices, y cada índice se divide en shards. Los shards primarios contienen la partición original de los datos, mientras que las réplicas son copias de esos shards que aportan redundancia y pueden contribuir a la disponibilidad y al procesamiento de consultas [8], [10].

Los nodos pueden desempeñar distintos roles (por ejemplo, maestro, de datos, de ingesta o coordinadores), lo que permite separar responsabilidades dentro del clúster [8]. En el experimento realizado se desplegaron tres nodos (`es01`, `es02` y `es03`) que conformaron un clúster funcional, lo cual permitió observar la distribución de shards y la respuesta ante fallos.

2.2. Comunicación entre componentes

Los clientes interactúan con Elasticsearch principalmente mediante su API REST sobre HTTP, mientras que los nodos se comunican internamente a través de la capa de transporte del clúster [7]. Cuando un cliente envía una solicitud, el nodo que la recibe puede actuar como nodo coordinador: determina qué shards deben participar, reenvía la operación a los nodos correspondientes y combina los resultados antes de responder [7], [10].

En las operaciones de escritura, la solicitud se dirige al shard primario correspondiente y luego se replica a las réplicas del shard, lo que implica coordinación entre nodos para mantener las copias actualizadas [10]. En las búsquedas, el sistema puede consultar copias del shard (primaria o réplicas), lo que favorece la distribución de la carga. Este modelo

refleja una comunicación cliente-servidor hacia el clúster y una coordinación interna entre nodos para ejecutar operaciones distribuidas.

2.3. Componentes principales de Apache Solr (SolrCloud)

En SolrCloud, los datos se organizan en colecciones que se dividen en shards, y cada shard puede tener múltiples réplicas. Una réplica actúa como líder del shard y coordina las actualizaciones, mientras que las demás réplicas mantienen copias del índice según su tipo [9]. SolrCloud soporta distintos tipos de réplicas (NRT, TLOG y PULL), que ofrecen diferentes compromisos entre capacidad de liderazgo, actualización y costo de recuperación [9], [11].

Las solicitudes distribuidas se enrutan a través del clúster para alcanzar los shards necesarios, y el sistema incorpora mecanismos para balanceo y tolerancia ante fallos en la atención de consultas [12]. Esta arquitectura lo hace comparable con Elasticsearch en términos de partición, replicación y coordinación distribuida.

2.4. Diagrama de arquitectura

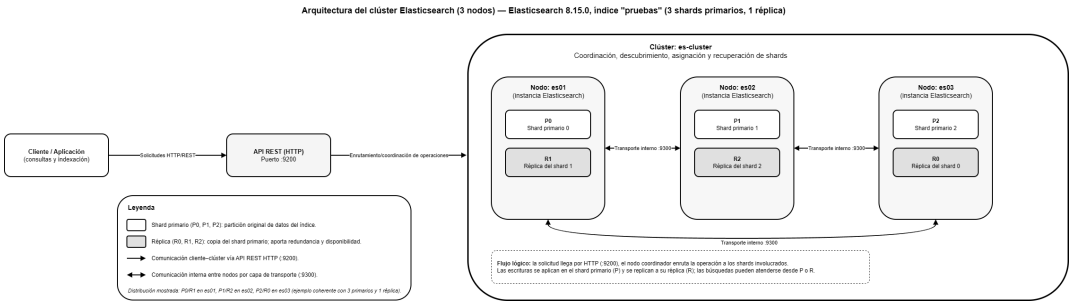


Figura 1: Arquitectura del clúster de Elasticsearch analizado (elaboración propia).

2.5. Evolución desde el esquema a mano (Anexo A)

El diagrama final se basó en un boceto inicial elaborado a mano (Anexo A). A partir de ese esquema se precisaron los elementos del clúster observados en la práctica: el cliente, el acceso HTTP al clúster, los tres nodos (`es01`, `es02` y `es03`), la distribución de shards primarios y réplicas, y la comunicación interna de transporte entre nodos. Esta evolución permitió pasar de una representación general a un diagrama más alineado con la arquitectura desplegada y analizada en la experimentación.

3. Análisis con los conceptos de la Unidad 1

3.1. Transparencias de distribución

Las transparencias de distribución ocultan aspectos de la naturaleza distribuida del sistema. A partir de la documentación oficial y de la experimentación realizada, se puede analizar su presencia en Elasticsearch y contrastarla con SolrCloud.

- **Transparencia de acceso:** Elasticsearch expone una interfaz HTTP/REST para interactuar con índices y documentos, lo que ofrece una forma uniforme de acceso para los clientes [7], [10]. En SolrCloud, las operaciones también se realizan mediante interfaces HTTP sobre colecciones y shards distribuidos [9], [12].
- **Transparencia de ubicación:** el cliente no necesita conocer en qué nodo específico se encuentra cada shard para realizar una consulta o indexación, ya que el clúster coordina el enrutamiento y la ejecución distribuida [10], [12]. Sin embargo, la ubicación física/lógica sigue siendo relevante para la administración del clúster.
- **Transparencia de migración:** los shards pueden reasignarse entre nodos (por ejemplo, ante cambios en el clúster o recuperación), lo que permite mover datos sin que el cliente gestione manualmente cada movimiento [8], [11], [13].
- **Transparencia de reubicación:** la reubicación de shards durante la operación del clúster es posible como parte de la gestión y recuperación, aunque puede tener impacto en rendimiento y estado del clúster; por ello, no es completamente invisible desde el punto de vista operativo [11], [13].
- **Transparencia de replicación:** Elasticsearch utiliza réplicas de shards para redundancia y soporte de disponibilidad/consultas [8], [10]. SolrCloud también emplea réplicas por shard, con tipos que definen su comportamiento y capacidad de liderazgo [9]. La replicación existe y aporta beneficios, aunque su configuración no es transparente para el administrador.
- **Transparencia de concurrencia:** múltiples clientes pueden operar sobre el sistema distribuido; el control de concurrencia y la coordinación de escrituras/lecturas se gestionan internamente en los mecanismos de shard primario, replicación y coordinación [9], [10]. No obstante, el comportamiento bajo concurrencia depende de la configuración y del tipo de operación.
- **Transparencia de fallos:** ambos sistemas incorporan mecanismos para tolerar fallos (réplicas, elección/gestión de líderes y recuperación), lo que puede ocultar ciertos fallos al cliente [11], [12], [13]. En el experimento, la caída de `es02` no impidió que el clúster mantuviera shards activos, aunque el evento sí fue observable a nivel administrativo.

3.2. Teorema CAP y extensión PACELC

3.2.1. Comportamiento ante partición de red (CAP)

Ante una partición de red, un sistema distribuido debe enfrentar el compromiso entre consistencia y disponibilidad. En Elasticsearch, la disponibilidad y la integridad del clúster dependen de mecanismos como el descubrimiento de nodos, la elección de nodo maestro y las reglas de asignación/recuperación de shards; configuraciones inadecuadas o la pérdida de quórum pueden afectar la capacidad del clúster para operar correctamente [13], [14]. En SolrCloud, la coordinación del clúster y la gestión de líderes/réplicas también influyen en el comportamiento ante fallos y particiones, y los procesos de recuperación buscan restablecer el estado operativo de los shards [9], [11], [12].

En términos prácticos, tanto Elasticsearch como SolrCloud priorizan la continuidad operativa mediante replicación y mecanismos de recuperación, pero el resultado ante una partición depende de la configuración, del número de nodos elegibles y del estado de las réplicas. Por ello, la clasificación CAP no debe asumirse de forma simplista sin considerar la configuración específica del clúster [11], [13].

3.2.2. Comportamiento sin partición (PACELC)

Cuando no existe partición, el compromiso se desplaza hacia latencia y consistencia. En Elasticsearch, las operaciones de escritura se procesan en el shard primario y se propagan a las réplicas, lo que introduce coordinación y posibles costos de latencia según la configuración y el estado del clúster [10], [13]. En búsquedas, la posibilidad de consultar réplicas puede favorecer la distribución de carga y la latencia de respuesta.

En SolrCloud, los tipos de réplica (NRT, TLOG y PULL) implican diferentes equilibrios: algunas réplicas pueden participar en liderazgo y mantener información para recuperación, mientras que otras priorizan la reducción de costo de indexación local a cambio de mayor dependencia de la replicación desde el líder [9], [11]. Esto muestra un compromiso entre latencia, frescura de datos y complejidad de recuperación, coherente con el enfoque de PACELC.

3.3. Modelos de comunicación

El modelo predominante en la interacción externa es **cliente-servidor**: los clientes envían solicitudes al clúster (Elasticsearch o SolrCloud) y reciben respuestas a través de interfaces HTTP [7], [12]. Internamente, la comunicación se da entre nodos del clúster para coordinar descubrimiento, elección de roles/liderazgo, enrutamiento de solicitudes, replicación y recuperación [9], [10], [11], [14].

Respecto al modo de comunicación, las solicitudes cliente-clúster suelen manejarse como interacciones petición-respuesta; sin embargo, dentro del clúster existen procesos

coordinados que pueden involucrar esperas y confirmaciones (por ejemplo, propagación de escrituras o procesos de recuperación) según la operación y configuración [10], [11]. Por ello, el sistema combina un acceso externo tipo cliente-servidor con mecanismos internos de coordinación distribuida.

3.4. Tolerancia a fallos

3.4.1. Fallos soportados, detección y recuperación

En Elasticsearch, la tolerancia a fallos se apoya en la replicación de shards y en los mecanismos de gestión del clúster. La elección y estabilidad del nodo maestro, el descubrimiento de nodos y la asignación de shards son elementos clave para detectar cambios en la membresía del clúster y responder ante fallos [8], [13], [14]. Cuando un nodo deja de estar disponible, el clúster puede reasignar réplicas o promover copias para mantener la disponibilidad de los shards, dependiendo de la configuración y del número de réplicas existentes.

En SolrCloud, la tolerancia a fallos se relaciona con la gestión de líderes de shard, la replicación y los procesos de recuperación. Los distintos tipos de réplicas (NRT, TLOG y PULL) influyen en la capacidad de asumir liderazgo y en la forma de recuperarse tras una falla [9], [11]. Además, la distribución de solicitudes permite continuar atendiendo consultas mientras existan réplicas disponibles [12].

3.4.2. Evidencia desde la experimentación

En la prueba realizada, el clúster de tres nodos mantuvo el estado *green* tras la detención de `es02`, con 2 nodos activos y 6 shards activos (0 sin asignar). Esto indica que las copias necesarias estaban disponibles en los nodos restantes. Al reiniciar `es02`, el nodo se reincorporó y el clúster redistribuyó shards, recuperando la distribución entre los tres nodos. Esta observación es consistente con el uso de réplicas y reasignación automática como mecanismos de tolerancia a fallos [8], [13].

3.4.3. Incidente real documentado

Un caso real documentado es el incidente de Elastic Cloud ocurrido el 18 de enero de 2019 en la región AWS `eu-west-1`, reportado por Elastic. Según el informe, una alta carga en la capa de coordinación (ZooKeeper) y problemas en la reconexión/caché de clientes del proxy provocaron degradación y, posteriormente, indisponibilidad de los despliegues en la región durante un periodo aproximado de 20 minutos, dentro de un incidente de mayor duración [15]. El reporte describe que la inestabilidad en la coordinación afectó el enrutamiento hacia los clústeres y que la mitigación incluyó estabilizar la capa de coordinación y escalar/ajustar la capa de proxy; además, se plantearon mejoras para

reducir el acoplamiento y aumentar la resiliencia. Este incidente ilustra que, en sistemas distribuidos, los fallos o degradaciones en componentes de coordinación y control también pueden impactar la disponibilidad percibida por los usuarios [13], [15].

Como referencia adicional del lado de SolrCloud, el issue *SOLR-7844* documenta un problema en el que la expiración de sesión de ZooKeeper durante la elección de líder de shard podía conducir a múltiples líderes, evidenciando riesgos de coordinación en escenarios de fallo [9], [16].

4. Experimentación propia

La experimentación se realizó en un clúster local de Elasticsearch 8.15.0 con tres nodos (`es01`, `es02`, `es03`) desplegados mediante Docker Compose. Se ejecutaron pruebas de levantamiento, creación de índice, inserción de documentos, caída de un nodo y recuperación. Los comandos y resultados completos se incluyen en el repositorio.

4.1. Levantamiento del clúster

Se iniciaron los contenedores y se verificó el estado del clúster. Los resultados mostraron tres nodos activos, con `es02` como nodo maestro y estado del clúster *green* (`number_of_nodes: 3`, `number_of_data_nodes: 3`).

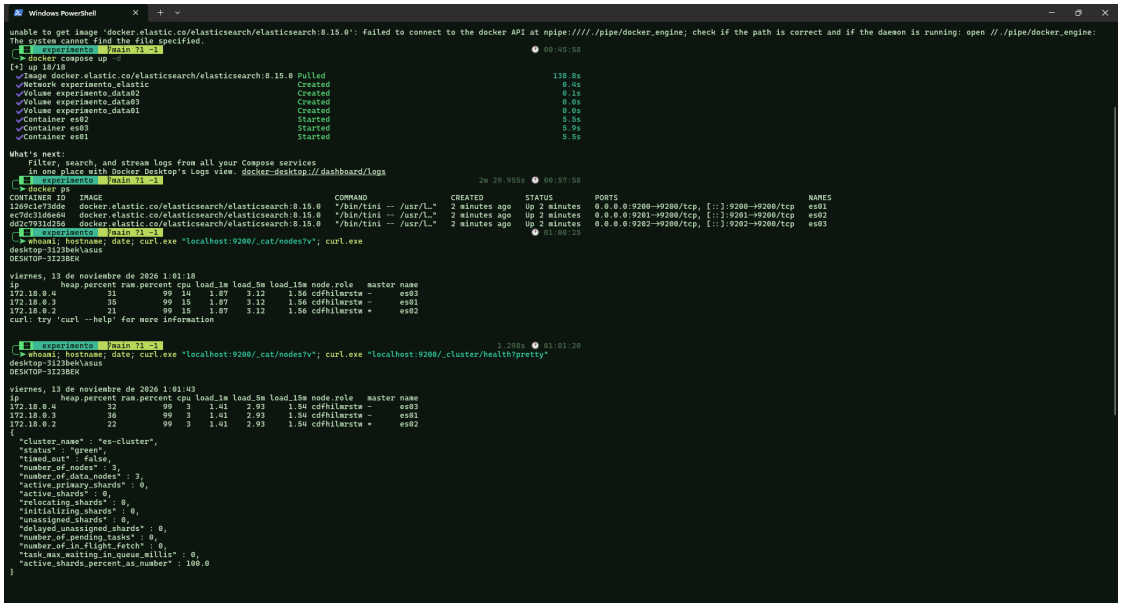


Figura 2: Levantamiento del clúster de Elasticsearch (captura propia).

Interpretación: El clúster se formó correctamente y alcanzó un estado saludable antes de crear índices. Esto evidencia la capacidad de Elasticsearch para coordinar múltiples nodos como un único sistema distribuido [7], [14].

4.2. Creación de índice y distribución de shards

Se creó el índice `pruebas` con 3 shards primarios y 1 réplica (`number_of_shards: 3`, `number_of_replicas: 1`), se insertaron 6 documentos y se ejecutó `_refresh`. La consulta `_cat/shards/pruebas?v` mostró 6 shards en estado `STARTED` (3 primarios y 3 réplicas), con distribución de documentos entre shards y réplicas ubicadas en nodos distintos a sus primarios.

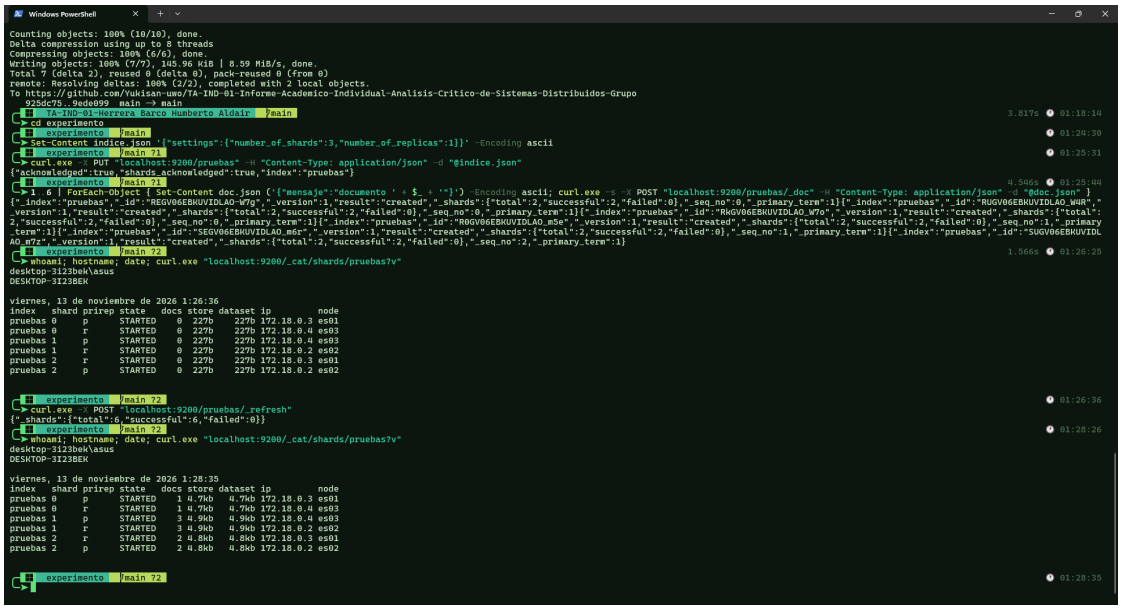


Figura 3: Distribución de shards primarios y réplicas del índice `pruebas` (captura propia).

Interpretación: La partición del índice en shards y la creación de réplicas permitieron distribuir datos y redundancia en el clúster. Esta configuración se relaciona con la transparencia de replicación y con la preparación del sistema para tolerar fallos de nodos [8], [10].

4.3. Caída de un nodo

Se detuvo el nodo `es02`. Posteriormente, `_cat/nodes` mostró 2 nodos activos (`es01` y `es03`) y `es03` asumió el rol de maestro. La salud del clúster se mantuvo en *green*, con `active_primary_shards: 3`, `active_shards: 6` y `unassigned_shards: 0`. La salida de shards evidenció que las copias se mantuvieron activas entre los nodos restantes.

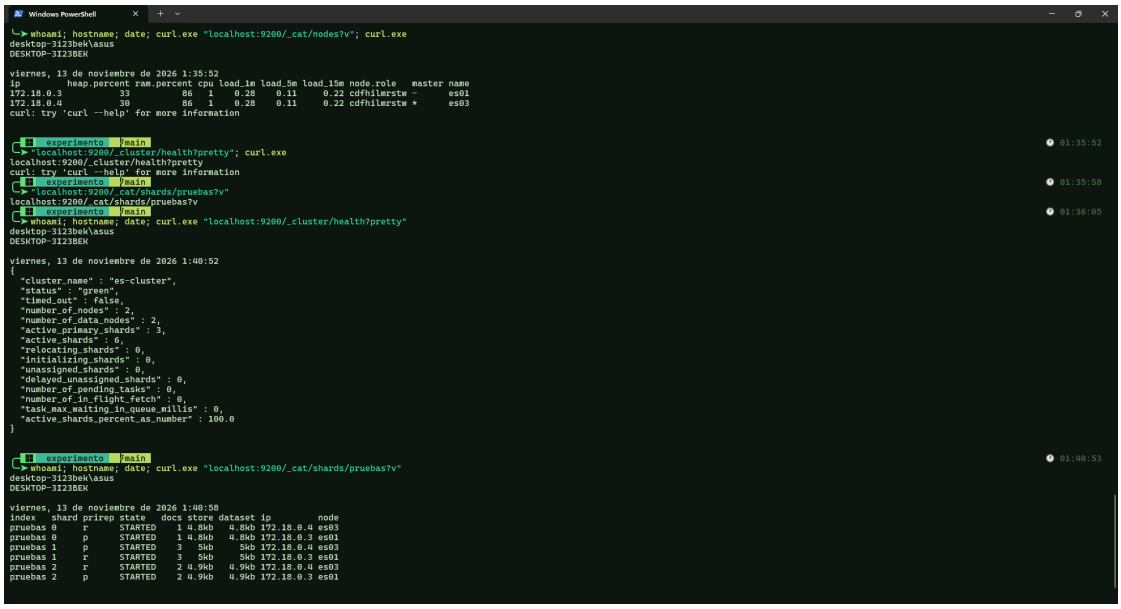


Figura 4: Estado del clúster tras la caída del nodo `es02` (captura propia).

Interpretación: La continuidad del estado *green* y la ausencia de shards sin asignar muestran que el clúster pudo reorganizar las copias disponibles para conservar el servicio. Este comportamiento refleja mecanismos de tolerancia a fallos basados en replicación y gestión del clúster [13], [14].

4.4. Recuperación del nodo

Al reiniciar `es02`, el nodo se reincorporó al clúster (3 nodos activos) y las réplicas se redistribuyeron, volviendo a ocupar nodos distintos según la asignación mostrada por `_cat/shards`. Los shards permanecieron en estado `STARTED` y se conservaron los mismos conteos de documentos.

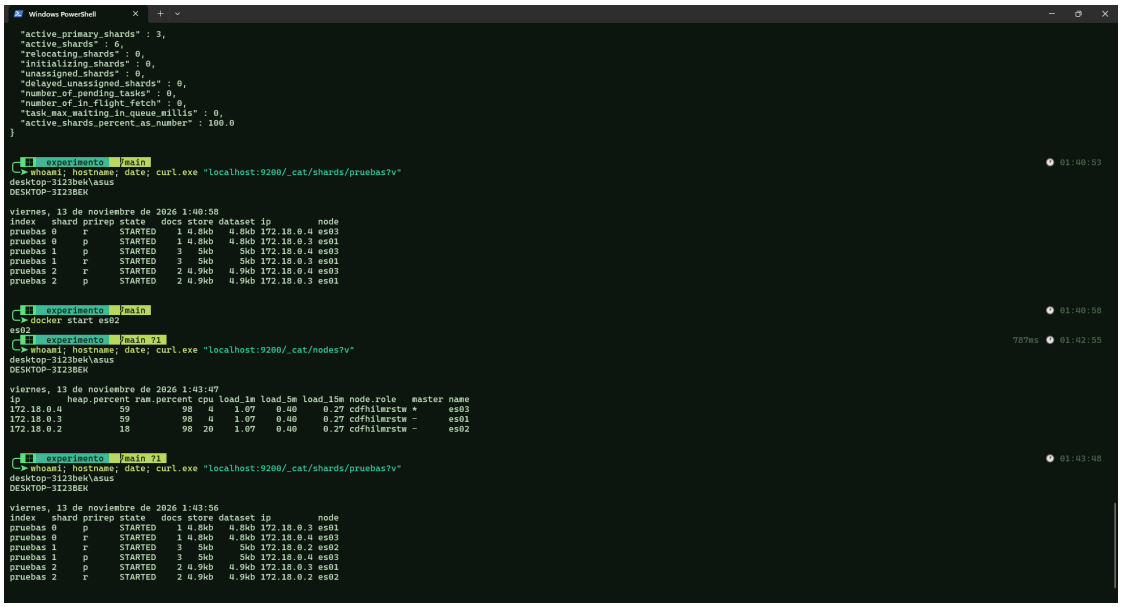


Figura 5: Recuperación y redistribución de shards tras reiniciar `es02` (captura propia).

Interpretación: La reincorporación del nodo permitió restablecer la distribución y la redundancia del clúster sin pérdida observable de datos en la prueba. Esto confirma la importancia de las réplicas y los procesos de recuperación para la disponibilidad del sistema [8], [13].

5. Ejemplos del entorno del estudiante

5.1. Ejemplo 1: [Completar: nombre del sistema observado]

Situación observada: [Completar: describe una situación concreta observada por ti (caída del servicio, demora en actualización, mensaje tardío, saldo que tardó en reflejarse, etc.)].

Relación con la Unidad 1: [Completar: relaciona la situación con conceptos como transparencia de fallos, consistencia eventual, disponibilidad, replicación, comunicación asíncrona o tolerancia a fallos].

5.2. Ejemplo 2: [Completar: nombre del sistema observado]

Situación observada: [Completar: describe otra situación concreta observada por ti].

Relación con la Unidad 1: [Completar: relaciona la situación con un concepto de la Unidad 1 y justifica brevemente].

6. Análisis comparativo: Elasticsearch vs. Apache Solr

La Tabla 1 compara Elasticsearch y Apache Solr (SolrCloud) utilizando criterios de la Unidad 1. La comparación se apoya en estudios académicos [1], [2], [3], [4], [5], [6] y en documentación técnica oficial [7], [8], [9], [10], [11], [12], [13], [14].

Cuadro 1: Comparación entre Elasticsearch y Apache Solr (SolrCloud)

Criterio (Unidad 1)	**Elasticsearch**	**Apache Solr (SolrCloud)**
Arquitectura y distribución	Clúster de nodos con índices particionados en shards primarios y réplicas; roles de nodos y coordinación distribuida para indexación/búsqueda [1], [7], [8].	Colecciones distribuidas en shards con réplicas; coordinación de clúster y liderazgo de shard para gestionar actualizaciones y consultas [9], [12].
Modelos de comunicación	Modelo cliente-servidor vía API HTTP/REST; comunicación interna entre nodos mediante la capa de transporte para descubrimiento, coordinación y replicación [7], [10], [14].	Modelo cliente-servidor vía HTTP/API; distribución y enrutamiento de solicitudes en el clúster, con coordinación entre nodos y componentes de gestión del clúster [9], [12].
Replicación y consistencia (CAP/PACELC)	Escrituras dirigidas al shard primario y propagadas a réplicas; los compromisos de consistencia, disponibilidad y latencia dependen de la configuración y del estado del clúster [10], [13].	Réplicas de shard con tipos NRT, TLOG y PULL que implican distintos compromisos entre actualización, capacidad de liderazgo, latencia y recuperación [9], [11].
Tolerancia a fallos y recuperación	Descubrimiento de nodos, elección de maestro, replicación y reasignación/recuperación de shards ante fallos; en el experimento el clúster mantuvo shards activos tras la caída de un nodo [8], [13], [14].	Liderazgo de shard, replicación y mecanismos de recuperación; la tolerancia a fallos depende del tipo y número de réplicas y de la coordinación del clúster. Incidentes de coordinación/liderazgo (p. ej. SOLR-7844) muestran riesgos a considerar [9], [11], [16].
Rendimiento y evidencia comparativa	El desempeño depende de configuración y carga; se reporta uso en escenarios de búsqueda/analítica distribuida [1], [2], [3].	Estudios y casos reportan buen desempeño y confiabilidad en búsqueda/indexación; comparaciones directas muestran resultados variables según escenario y configuración [4], [5], [6].

6.1. Discusión comparativa

Elasticsearch y SolrCloud comparten una base conceptual similar: ambos distribuyen índices en shards y utilizan réplicas para soportar consultas y fallos. Sin embargo, difieren en la forma de gestionar ciertos aspectos operativos y en los mecanismos disponibles para balancear consistencia, disponibilidad y rendimiento. La evidencia experimental mostró que Elasticsearch puede mantener la disponibilidad de los datos ante la caída de un nodo cuando existen réplicas asignadas correctamente. Por su parte, SolrCloud ofrece opciones explícitas de tipos de réplica (NRT, TLOG, PULL), lo que permite ajustar el comportamiento de indexación y recuperación según las necesidades [9], [11].

Los estudios comparativos sugieren que no existe un ganador absoluto: el rendimiento relativo depende del conjunto de datos, la configuración, el hardware y el tipo de consulta [3], [4]. Por ello, la selección entre Elasticsearch y Solr debe considerar no solo velocidad, sino también modelo de operación, tolerancia a fallos, consistencia requerida y capacidades de administración.

7. Postura crítica

7.1. Fortalezas

Elasticsearch presenta fortalezas importantes como sistema distribuido: escalabilidad horizontal mediante nodos y shards, replicación para disponibilidad, y una interfaz HTTP/REST que facilita la integración con aplicaciones [7], [8], [10]. En la experimentación, el clúster mantuvo la disponibilidad de los shards tras la caída de un nodo, lo que evidencia en la práctica el valor de la replicación y la reasignación automática. Además, existe un ecosistema y documentación amplia para su despliegue y observación [2], [13].

7.2. Limitaciones y compromisos (trade-offs)

El uso de Elasticsearch implica compromisos propios de los sistemas distribuidos. La configuración de shards, réplicas y nodos maestros influye en la resiliencia y el rendimiento: una configuración inadecuada puede afectar la disponibilidad o la recuperación ante fallos [8], [13]. Asimismo, la coordinación necesaria para escribir y replicar datos introduce costos de latencia y complejidad operativa [10]. Frente a particiones o fallos de componentes de coordinación, la disponibilidad puede verse comprometida, como lo ilustra el incidente de Elastic Cloud de 2019 [15]. En comparación, SolrCloud también enfrenta compromisos, pero los hace más explícitos en la elección de tipos de réplica y en su dependencia de mecanismos de coordinación [9], [11].

7.3. Contextos donde no sería la mejor opción

Elasticsearch no sería la mejor opción cuando se requiere un sistema transaccional relacional con garantías ACID tradicionales como requisito principal, cuando el volumen de datos es pequeño y no justifica la complejidad operativa de un clúster distribuido, o cuando las restricciones de recursos impiden sostener múltiples nodos y réplicas. En escenarios donde se prioricen configuraciones específicas de replicación/liderazgo o una integración particular con el ecosistema de Solr, Apache Solr podría ser una alternativa más adecuada [4], [9].

8. Conclusiones

1. La experimentación mostró que un clúster de Elasticsearch con réplicas puede mantener el estado *green* y conservar los shards activos ante la caída de un nodo, gracias a la reasignación de copias entre los nodos disponibles.

2. Elasticsearch y Apache Solr comparten principios de distribución (shards y réplicas), pero presentan diferencias en sus mecanismos de coordinación y en las opciones de configuración (por ejemplo, los tipos de réplica en SolrCloud), lo que implica compromisos distintos entre consistencia, disponibilidad, latencia y recuperación [9], [10], [11].

3. La tolerancia a fallos no depende únicamente de contar con réplicas: también influyen la elección de nodos maestros/líderes, la coordinación del clúster y la correcta configuración, como se observó en la práctica y en incidentes documentados [13], [14], [15], [16].

4. La elección entre Elasticsearch y Solr debe basarse en el contexto de uso, los requisitos no funcionales y la evidencia disponible; las comparaciones de rendimiento aportan criterios útiles, pero no reemplazan el análisis de arquitectura y operación distribuida [3], [4], [6].

9. Declaración de uso de inteligencia artificial

Herramienta: Claude (Anthropic).
Uso: La IA me ayudó a mejorar la redacción, organizar el contenido y escribir el código en LaTeX. También me apoyó en la traducción de los artículos, la documentación utilizada y en la realización del experimento, guiándome con el uso de comandos en la terminal y resolviendo dudas durante el proceso.
Verificación: Se revisaron todos los DOI uno por uno con ayuda de IA para traducir y comprender mejor los artículos. También se consultó documentación oficial de las

herramientas utilizadas para entender mejor los conceptos y mejorar la elaboración del documento.

Referencias

[1] S. Cheresharov, G. Gustinov, V. Tabakova-Komsalova, I. Stoyanov y P. Georgiev, «Low-Resource SLM Models for Elastic Semantic Search,» *Electronics*, vol. 15, n.º 18, pág. 4056, 8 de sep. de 2026. DOI: 10.3390/electronics15184056 dirección: https://doi.org/10.3390/electronics15184056

[2] G. Calderon, G. Del Campo, E. Saavedra y A. Santamaría, «Monitoring Framework for the Performance Evaluation of an IoT Platform with Elasticsearch and Apache Kafka,» *Information Systems Frontiers*, vol. 26, n.º 6, págs. 2373-2389, 13 de jul. de 2023. DOI: 10.1007/s10796-023-10409-2 dirección: https://doi.org/10.1007/s10796-023-10409-2

[3] I. Feraru e I. Hrubaru, «Decomposing Engine and Schema Effects in Database Performance: A Result-Equivalent E-Commerce Benchmark,» *Electronics*, vol. 15, n.º 22, pág. 5209, 10 de nov. de 2026. DOI: 10.3390/electronics15225209 dirección: https://doi.org/10.3390/electronics15225209

[4] A. Deniz, M. M. Elömer y A. A. Aydin, «A comparison of Apache Solr and Elasticsearch technologies in support of large-scale data analysis,» *Gümüşhane Üniversitesi Fen Bilimleri Enstitüsü Dergisi*, 10 de mar. de 2023. DOI: 10.17714/gumusfenbil.1213317 dirección: https://doi.org/10.17714/gumusfenbil.1213317

[5] L. Lu, B. Yang y X. Wang, «Construction of a data-driven knowledge discovery path from the perspective of intelligent academic evaluation: an empirical study based on institutional repository and solr technology,» *Scientific Reports*, vol. 16, n.º 1, 13 de abr. de 2026. DOI: 10.1038/s41598-026-47818-7 dirección: https://doi.org/10.1038/s41598-026-47818-7

[6] S. Unger, C. K. Raak y T. Ostermann, «Reliability and Performance of the Online Literature Database CAMbase after Changing from a Semantic Search to a Score Ranking Algorithm,» *SN Computer Science*, vol. 4, n.º 5, 9 de sep. de 2023. DOI: 10.1007/s42979-023-02146-9 dirección: http://dx.doi.org/10.1007/s42979-023-02146-9

[7] Elastic. «Distributed architecture.» Elastic Documentation, visitado 13 de nov. de 2026. dirección: https://www.elastic.co/docs/deploy-manage/distributed-architecture

[8] Elastic. «Clusters, nodes, and shards.» Elasticsearch Documentation, visitado 13 de nov. de 2026. dirección: https://www.elastic.co/docs/deploy-manage/distributed-architecture/clusters-nodes-shards

[9] The Apache Software Foundation. «Shards and Indexing Data in SolrCloud.» Apache Solr Reference Guide, visitado 13 de nov. de 2026. dirección: https://solr.apache.org/guide/solr/latest/deployment-guide/solrcloud-shards-indexing.html

[10] Elastic. «Reading and writing documents.» Elasticsearch Documentation, visitado 13 de nov. de 2026. dirección: https://www.elastic.co/docs/deploy-manage/distributed-architecture/reading-and-writing-documents

[11] The Apache Software Foundation. «SolrCloud Recoveries and Write Tolerance.» Apache Solr Reference Guide, visitado 13 de nov. de 2026. dirección: https://solr.apache.org/guide/solr/latest/deployment-guide/solrcloud-recoveries-and-write-tolerance.html

[12] The Apache Software Foundation. «Distributed Requests.» Apache Solr Reference Guide, visitado 13 de nov. de 2026. dirección: https://solr.apache.org/guide/solr/latest/deployment-guide/solrcloud-distributed-requests.html

[13] Elastic. «Availability and resilience in Elasticsearch.» Elastic Documentation, visitado 13 de nov. de 2026. dirección: https://www.elastic.co/docs/deploy-manage/production-guidance/availability-and-resilience

[14] Elastic. «Discovery and cluster formation.» Elasticsearch Documentation, visitado 13 de nov. de 2026. dirección: https://www.elastic.co/docs/deploy-manage/distributed-architecture/discovery-cluster-formation

[15] P. Moustafellos, U. Cohen y S. Wallez. «Elastic Cloud January 18, 2019 Incident Report.» Elastic Blog, visitado 13 de nov. de 2026. dirección: https://www.elastic.co/blog/elastic-cloud-january-18-2019-incident-report

[16] Apache Software Foundation. «SOLR-7844: Zookeeper session expiry during shard leader election can cause multiple leaders.» Apache Jira Issue, visitado 13 de nov. de 2026. dirección: https://issues.apache.org/jira/browse/SOLR-7844

A. Esquema inicial a mano de la arquitectura

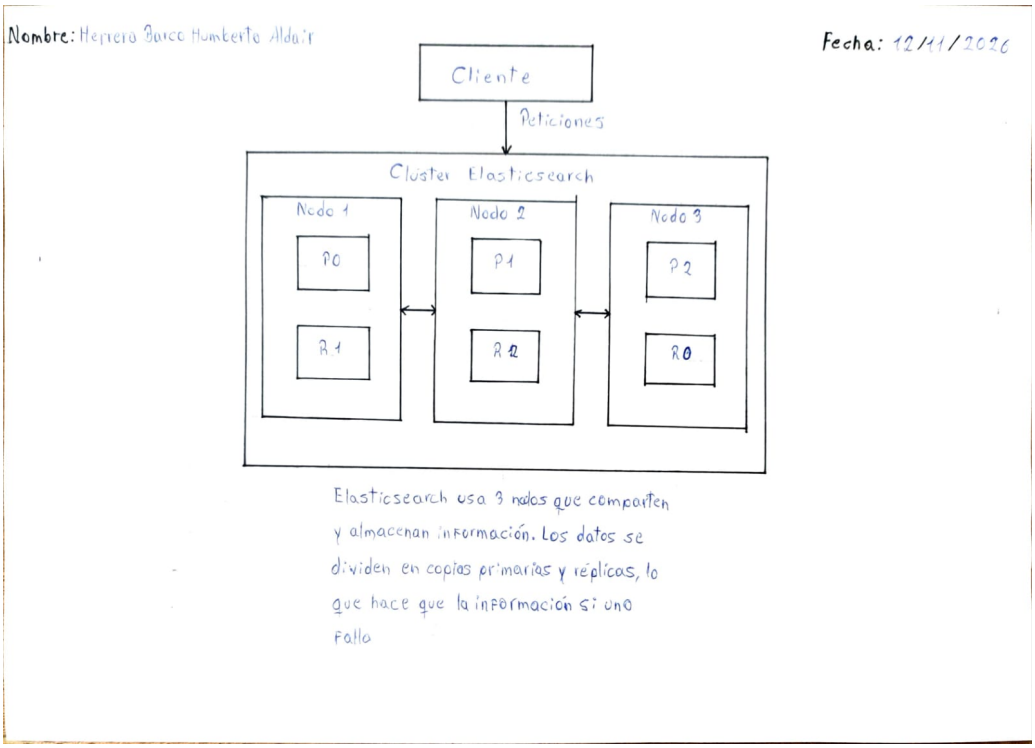


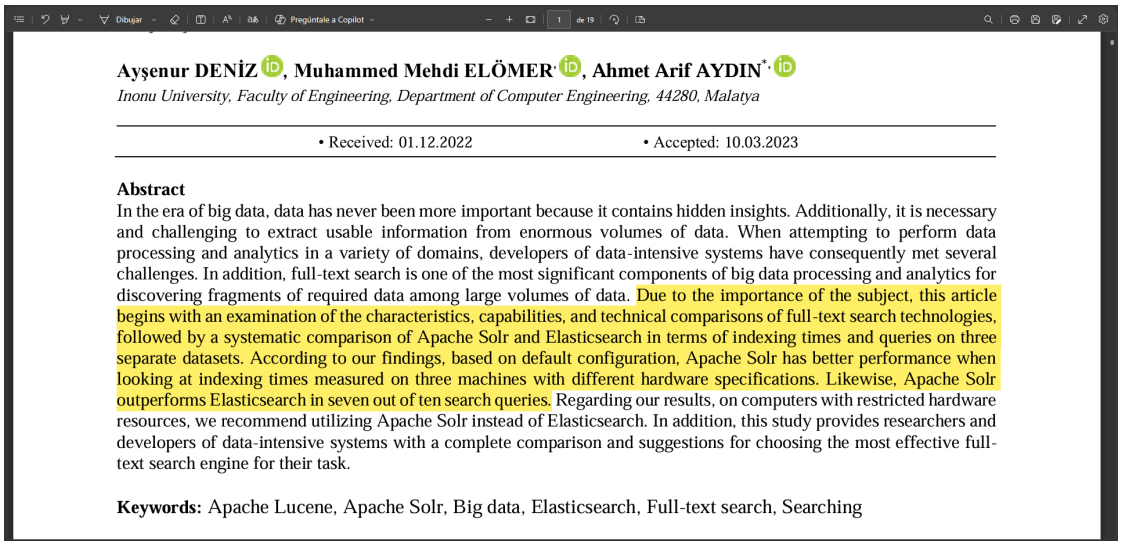
Figura 6: Esquema inicial a mano de la arquitectura del clúster (elaboración propia).

Nota: verificar que en la imagen se aprecien nombre y fecha, según los requisitos de la actividad.

B. Capturas de lectura de las fuentes

Las siguientes capturas corresponden a fragmentos de los artículos citados en el informe. Cada imagen incluye el pasaje subrayado/anotado y el número de página visible en el PDF.

B.1. Fuente 1: Comparación de Elasticsearch y Apache Solr [4]



Ayşenur DENİZ, Muhammed Mehdi ELÖMER, Ahmet Arif AYDIN
Inonu University, Faculty of Engineering, Department of Computer Engineering, 44280, Malatya

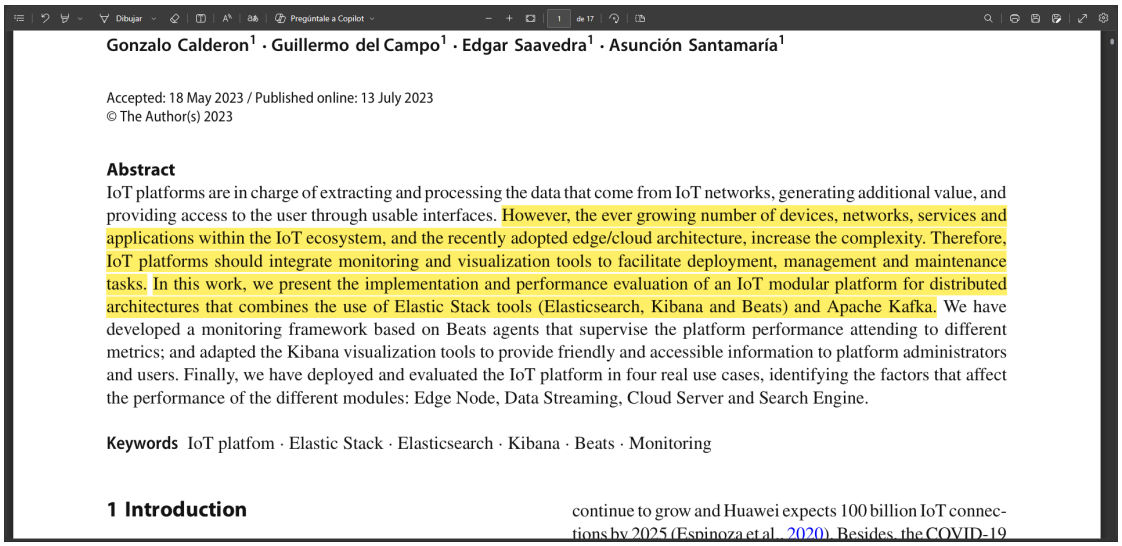
• Received: 01.12.2022 • Accepted: 10.03.2023

Abstract
In the era of big data, data has never been more important because it contains hidden insights. Additionally, it is necessary and challenging to extract usable information from enormous volumes of data. When attempting to perform data processing and analytics in a variety of domains, developers of data-intensive systems have consequently met several challenges. In addition, full-text search is one of the most significant components of big data processing and analytics for discovering fragments of required data among large volumes of data. Due to the importance of the subject, this article begins with an examination of the characteristics, capabilities, and technical comparisons of full-text search technologies, followed by a systematic comparison of Apache Solr and Elasticsearch in terms of indexing times and queries on three separate datasets. According to our findings, based on default configuration, Apache Solr has better performance when looking at indexing times measured on three machines with different hardware specifications. Likewise, Apache Solr outperforms Elasticsearch in seven out of ten search queries. Regarding our results, on computers with restricted hardware resources, we recommend utilizing Apache Solr instead of Elasticsearch. In addition, this study provides researchers and developers of data-intensive systems with a complete comparison and suggestions for choosing the most effective full-text search engine for their task.

Keywords: Apache Lucene, Apache Solr, Big data, Elasticsearch, Full-text search, Searching

Figura 7: Lectura anotada de Deniz et al. (2023), utilizada como respaldo de la comparación entre Elasticsearch y Apache Solr [4].

B.2. Fuente 2: Plataforma distribuida con Elasticsearch y Kafka [2]



Gonzalo Calderon[1] · Guillermo del Campo[1] · Edgar Saavedra[1] · Asunción Santamaría[1]

Accepted: 18 May 2023 / Published online: 13 July 2023
© The Author(s) 2023

Abstract
IoT platforms are in charge of extracting and processing the data that come from IoT networks, generating additional value, and providing access to the user through usable interfaces. However, the ever growing number of devices, networks, services and applications within the IoT ecosystem, and the recently adopted edge/cloud architecture, increase the complexity. Therefore, IoT platforms should integrate monitoring and visualization tools to facilitate deployment, management and maintenance tasks. In this work, we present the implementation and performance evaluation of an IoT modular platform for distributed architectures that combines the use of Elastic Stack tools (Elasticsearch, Kibana and Beats) and Apache Kafka. We have developed a monitoring framework based on Beats agents that supervise the platform performance attending to different metrics; and adapted the Kibana visualization tools to provide friendly and accessible information to platform administrators and users. Finally, we have deployed and evaluated the IoT platform in four real use cases, identifying the factors that affect the performance of the different modules: Edge Node, Data Streaming, Cloud Server and Search Engine.

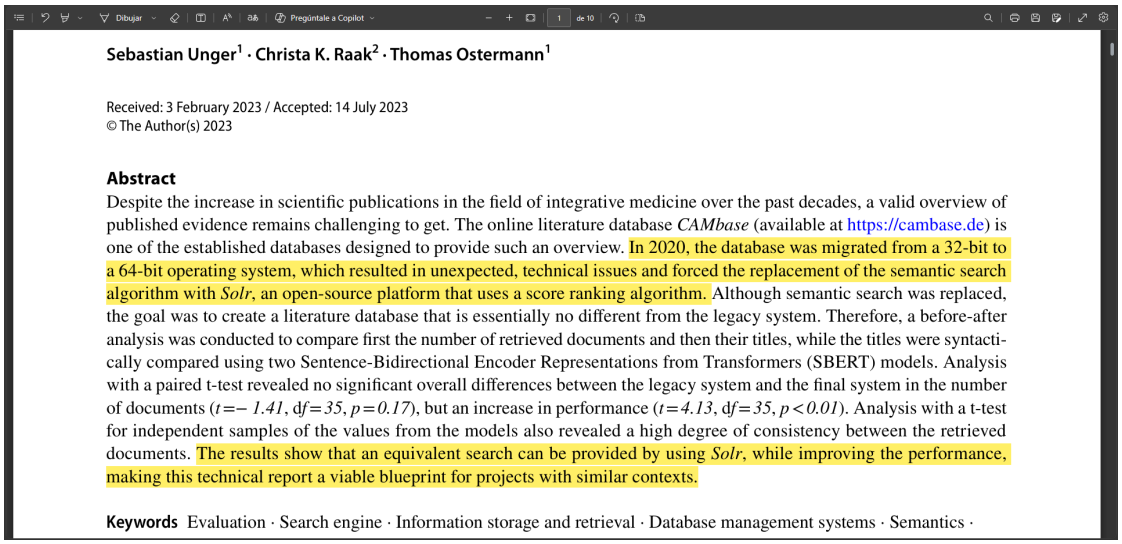
Keywords IoT platfom · Elastic Stack · Elasticsearch · Kibana · Beats · Monitoring

1 Introduction

continue to grow and Huawei expects 100 billion IoT connections by 2025 (Espinoza et al., 2020). Besides, the COVID-19

Figura 8: Lectura anotada de Calderon et al. (2023), utilizada como respaldo del uso de Elasticsearch en arquitecturas distribuidas [2].

B.3. Fuente 3: Uso de Solr y evaluación de rendimiento [6]



Sebastian Unger[1] · Christa K. Raak[2] · Thomas Ostermann[1]

Received: 3 February 2023 / Accepted: 14 July 2023
© The Author(s) 2023

Abstract
Despite the increase in scientific publications in the field of integrative medicine over the past decades, a valid overview of published evidence remains challenging to get. The online literature database *CAMbase* (available at https://cambase.de) is one of the established databases designed to provide such an overview. In 2020, the database was migrated from a 32-bit to a 64-bit operating system, which resulted in unexpected, technical issues and forced the replacement of the semantic search algorithm with *Solr*, an open-source platform that uses a score ranking algorithm. Although semantic search was replaced, the goal was to create a literature database that is essentially no different from the legacy system. Therefore, a before-after analysis was conducted to compare first the number of retrieved documents and then their titles, while the titles were syntactically compared using two Sentence-Bidirectional Encoder Representations from Transformers (SBERT) models. Analysis with a paired t-test revealed no significant overall differences between the legacy system and the final system in the number of documents ($t = -1.41$, $df = 35$, $p = 0.17$), but an increase in performance ($t = 4.13$, $df = 35$, $p < 0.01$). Analysis with a t-test for independent samples of the values from the models also revealed a high degree of consistency between the retrieved documents. The results show that an equivalent search can be provided by using *Solr*, while improving the performance, making this technical report a viable blueprint for projects with similar contexts.

Keywords Evaluation · Search engine · Information storage and retrieval · Database management systems · Semantics ·

Figura 9: Lectura anotada de Unger et al. (2023), utilizada como respaldo del uso y rendimiento de Apache Solr [6].